

[CS230] Sequence Models: Recurrent Neural Networks (RNN)

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-9. Deep Learning Fundamentals & CNNs - *Completed*

Chapter 10. Sequence Models (Current Part)

- 10.1 Recurrent Neural Networks (RNN)
 - * Why Sequence Models? (Time & Order)
 - * RNN Architecture (Unrolled View)
 - * Forward Propagation Formulas
 - * BPTT (Backpropagation Through Time)
- 10.2 GRU & LSTM (Gated Units) - *Upcoming*
- 10.3 NLP & Word Embeddings - *Upcoming*

지난 시간 복습 및 연결

우리는 지금까지 고정된 크기의 이미지($H \times W$)를 처리하는 CNN을 다뤘습니다. 하지만 세상의 많은 데이터는 '순서(Sequence)'와 '시간(Time)'을 가지고 있습니다. "나는 프랑스에 가서..."라는 말을 들으면, 뒤에 "프랑스어"라는 말이 나올 확률이 높다는 것을 우리는 압니다. 이는 앞선 단어들의 문맥(Context)을 기억하기 때문입니다. 기존 신경망은 이 '기억' 능력이 없습니다. 오늘은 기억을 가진 신경망, RNN의 세계로 들어갑니다.

1 Unit Overview

□ 핵심 목표

이 단원은 시계열 데이터를 처리하는 RNN의 기본 원리와 학습 알고리즘을 다룹니다.

- 구조: 은닉 상태(Hidden State)를 통해 과거 정보를 현재로 전달하는 루프 구조를 이해합니다.
- 수식: $a^{(t)} = g(W_{aa}a^{(t-1)} + W_{ax}x^{(t)})$ 공식을 마스터합니다.
- 공유: 모든 시간 단계에서 파라미터(W)를 공유하여 일반화 성능을 높이는 원리를 파악합니다.
- 학습: 시간을 거슬러 올라가는 역전파, BPTT의 개념을 익힙니다.

2 Essential Terminology

기호	용어	설명
$x^{(t)}$	Input	시간 t 에서의 입력 (예: t 번째 단어).
$a^{(t)}$	Hidden State	시간 t 에서의 은닉 상태. (기억)
$y^{(t)}$	Output	시간 t 에서의 출력 (예: 다음 단어 예측).
W_{aa}	Weight (Hidden)	과거 기억을 현재로 가져오는 가중치.
W_{ax}	Weight (Input)	현재 입력을 받아들이는 가중치.

3 Core Concepts: 순환의 마법

3.1 1. RNN Architecture (Unrolled)

RNN은 자신을 가리키는 화살표(Loop)를 가집니다. 이를 시간축으로 펼치면 다음과 같습니다.

- **입력:** 매 시점 t 마다 $x^{(t)}$ 가 들어옵니다.
- **전달:** 이전 시점의 기억 $a^{(t-1)}$ 이 현재 시점 t 로 전달됩니다.
- **출력:** 두 정보를 합쳐서 $y^{(t)}$ 를 출력하고, 다음 시점 $t+1$ 로 기억 $a^{(t)}$ 를 넘깁니다.

3.2 2. Forward Propagation Formulas

RNN의 핵심은 "현재 입력과 과거 기억을 섞어서 새로운 기억을 만든다"는 것입니다.

□ 은닉 상태 업데이트 (기억 갱신) (수학적 정의)

$$a^{(t)} = \tanh(W_{aa}a^{(t-1)} + W_{ax}x^{(t)} + b_a)$$

- $W_{aa}a^{(t-1)}$: 과거의 기억 반영.
- $W_{ax}x^{(t)}$: 현재의 정보 반영.
- $\tanh$: 값을 -1 ~ 1 사이로 압축하여 폭발 방지 (주로 사용).

□ 출력 계산 (수학적 정의)

$$\hat{y}^{(t)} = \text{softmax}(W_{ya}a^{(t)} + b_y)$$

- 현재의 기억($a^{(t)}$)을 바탕으로 예측을 수행합니다.

Key Point (Parameter Sharing): $t = 1$ 이든 $t = 100$ 이든, W_{aa}, W_{ax}, W_{ya} 는 모두 똑같은 행렬을 재사용합니다. 이것이 RNN이 길이 제한 없이 문장을 처리할 수 있는 비결입니다.

4 Deep Dive: Backpropagation Through Time (BPTT)

RNN의 학습은 시간을 거슬러 올라갑니다.

- **Loss:** 전체 손실 L 은 각 시간 단계별 손실의 합입니다. $L = \sum L^{(t)}$.
- **Gradient:** $t = 100$ 시점의 오차를 수정하려면, $t = 99, 98, \dots, 1$ 시점의 상태까지 영향을 미쳐야 합니다.

- **Problem:** 미분값이 계속 곱해지면서 (W_{aa}^{100}), 값이 0으로 사라지거나 (Vanishing) 무한대로 커지는 (Exploding) 문제가 발생합니다. 이로 인해 기본 RNN은 장기 의존성 (Long-term Dependency)을 학습하기 어렵습니다.

5 Implementation: RNN Step & Loop

NumPy로 RNN의 내부 동작을 구현해 봅시다.

```

1 import numpy as np
2
3 def rnn_cell_forward(xt, a_prev, parameters):
4     """
5     단일 타임 스텝 (t) 처리
6     """
7     Wax = parameters["Wax"]
8     Waa = parameters["Waa"]
9     Wya = parameters["Wya"]
10    ba = parameters["ba"]
11    by = parameters["by"]
12
13    # 1. 다음 은닉 상태 계산 (Tanh 사용)
14    # a_next = tanh(Waa*a_prev + Wax*xt + ba)
15    a_next = np.tanh(np.dot(Waa, a_prev) + np.dot(Wax, xt) + ba)
16
17    # 2. 현재 출력 예측 (Softmax 가정)
18    yt_pred = softmax(np.dot(Wya, a_next) + by)
19
20    return a_next, yt_pred
21
22 def rnn_forward(x, a0, parameters):
23     """
24     전체 시퀀스 처리 (Time Loop)
25     """
26    n_x, m, T_x = x.shape # T_x: 시간 길이 (Timesteps)
27    n_y, n_a = parameters["Wya"].shape
28
29    a = np.zeros((n_a, m, T_x)) # 모든 기억 저장용
30    y_pred = np.zeros((n_y, m, T_x)) # 모든 출력 저장용
31
32    a_next = a0 # 초기 기억
33
34    # 시간 순서대로 루프 (의RNN 핵심)
35    for t in range(T_x):
36        xt = x[:, :, t] # 번째 t 입력
37
38        # 셀 업데이트
39        a_next, yt_pred = rnn_cell_forward(xt, a_next, parameters)
40
41        # 저장
42        a[:, :, t] = a_next
43        y_pred[:, :, t] = yt_pred
44
45    return a, y_pred
46
47 def softmax(x):
48    e_x = np.exp(x - np.max(x))
49    return e_x / e_x.sum(axis=0)

```

Listing 1: RNN Forward Pass Implementation

6 FAQ & Pitfalls

Q. 왜 ReLU 대신 Tanh를 쓰나요?

A. RNN은 같은 가중치를 수십, 수백 번 반복해서 곱합니다. ReLU를 쓰면 값이 계속 커져서 발산(Exploding)하기 쉽습니다. Tanh는 값을 -1 1 사이로 묶어두어(Bounding) 안정적인 학습을 돕습니다.

Q. RNN은 병렬 처리가 안 되나요?

A. 네, 구조적으로 어렵습니다. t 시점의 계산을 하려면 반드시 $t - 1$ 시점의 결과가 나와야 하기 때문입니다(Sequential). 이것이 트랜스포머(Transformer)가 등장하게 된 배경 중 하나입니다.

다음 단계 (Next Step)

기본 RNN은 이론적으로 훌륭하지만, 문장이 조금만 길어져도(10단어 이상) 앞부분 내용을 까먹는 기울기 소실 문제가 있습니다.

이를 해결하기 위해 딥러닝 연구자들은 "기억을 얼마나 오래 유지할지 스스로 결정하는 게이트(Gate)"를 만들었습니다. 다음 시간에는 현대 NLP의 근간이 된 [GRU (Gated Recurrent Unit)]와 [LSTM (Long Short-Term Memory)]에 대해 다루겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Structure:** 입력(x) + 이전 기억(a) $\rightarrow$ 새 기억 $\rightarrow$ 출력(y).
2. **Sharing:** 모든 시점에서 동일한 파라미터(W_{ax}, W_{aa})를 쓴다.
3. **Formula:** $a^{(t)} = \tanh(W_{aa}a^{(t-1)} + W_{ax}x^{(t)} + b_a)$.
4. **Limit:** 긴 시퀀스에서는 기울기가 소실되어(Vanishing Gradient) 초기 기억을 잃는다.